```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import "@openzeppelin/contracts/utils/math/SafeMath.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

import "hardhat/console.sol"; //to console output

contract TokenMarketPlace is Ownable {
    using SafeERC20 for IERC20;
    //jitne bhi IERC20 type ke variable honge
    //unke pass ab SafeERC20 ke method available ho jayenge

    using SafeMath for uint256; //x.add(y) = x+y

    uint256 public tokenPrice = 2e16 wei; // 0.02 ether per GLD token //2 * 10^16
    //10 * 0.02 = 0.2 ether

    uint256 public sellerCount = 1;
    uint256 public buyerCount = 1;
    uint256 public prevAdjustedRatio;
    IERC20 public gldToken;
    /*
    IERC20
    IERC20 is an interface from OpenZeppelin's ERC-20 contract, which defines the
standard functions of an ERC-20 token.
    It allows the smart contract to interact with any ERC-20 token without knowing
its implementation.

    gldToken
    This is the ERC-20 token contract that will be used in this marketplace.
    It holds the reference to the token's smart contract, which allows this
marketplace contract to transfer, buy, and sell tokens.

    IERC20 public gldToken; is a reference to the ERC-20 token contract, allowing
this marketplace contract to buy, sell, and manage GLD tokens safely.

    */
```

```
    event TokenPriceUpdated(uint256 newPrice);
    event TokenBought(address indexed buyer, uint256 amount, uint256 totalCost);
    event TokenSold(
        address indexed seller,
        uint256 amount,
        uint256 totalEarned
    );
    event TokensWithdrawn(address indexed owner, uint256 amount);
    event EtherWithdrawn(address indexed owner, uint256 amount);
    event CalculateTokenPrice(uint256 priceToPay);

    constructor(address _gldToken) Ownable(msg.sender) {
        //apne parent ke constructor ko call karna hai to ---name of parent
constructor and pass the value
        gldToken = IERC20(_gldToken);
        /* Initialized in Constructor:
_gldToken is the address of an existing ERC-20 token contract.
IERC20(_gldToken) casts this address as an IERC20 interface, allowing this contract
to call ERC-20 functions.

----------------Simple Meaning with a Life Example
Think of IERC20 public gldToken; as a wallet app that supports different bank cards
(like Visa, Mastercard).

IERC20 → Just like all bank cards follow a common standard (chip, PIN, swipe/tap),
all ERC-20 tokens follow a common standard (transfer, balance check).
gldToken → This is like one specific bank card that the wallet app is currently
using.
Life Example: Wallet & Bank Card
📌 Imagine you have a wallet app that can store different bank cards (HDFC, ICICI,
SBI).
📌 Right now, your wallet is linked to an HDFC debit card.


IERC20 public hdfcCard;
This means the wallet app (contract) knows how to use the HDFC card (token contract)
for transactions.
```

```
Just like you can check balance, send money, or receive money using your card, this
contract can check token balance, transfer, and receive tokens using gldToken.

✅ Conclusion: IERC20 public gldToken; means this contract is linked to a specific
ERC-20 token (like a wallet linked to a bank card), so it can perform transactions
with it. 🚀

*/
    }

    // Updated logic for token price calculation with safeguards
    function adjustTokenPriceBasedOnDemand() public {
        uint256 marketDemandRatio = buyerCount.mul(1e18).div(sellerCount);
        uint256 smoothingFactor = 1e18;
        uint256 adjustedRatio = marketDemandRatio.add(smoothingFactor).div(2);
        if (prevAdjustedRatio != adjustedRatio) {
            prevAdjustedRatio = adjustedRatio;
            uint256 newTokenPrice = tokenPrice.mul(adjustedRatio).div(1e18);
            uint256 minimumPrice = 2e16;
            if (newTokenPrice < minimumPrice) {
                tokenPrice = minimumPrice;
            }
            tokenPrice = newTokenPrice;
        }
    }

    // Buy tokens from the marketplace
    function buyGLDToken(uint256 _amountOfToken) public payable {
        require(_amountOfToken > 0, "Invalid Token amount");

        uint256 requiredTokenPrice = calculateTokenPrice(_amountOfToken);
        console.log("requiredTokenPrice", requiredTokenPrice);
        require(requiredTokenPrice == msg.value, "Incorrect token price paid");
        buyerCount = buyerCount + 1;
        // Transfer token to the buyer address
        gldToken.safeTransfer(msg.sender, _amountOfToken);
        // Event Emiting
        emit TokenBought(msg.sender, _amountOfToken, requiredTokenPrice);
    }
```

```
    function calculateTokenPrice(uint256 _amountOfToken)
        public
        returns (uint256)
    {
        require(_amountOfToken > 0, "Amount Of Token > 0");
        adjustTokenPriceBasedOnDemand();
        uint256 amountToPay = _amountOfToken.mul(tokenPrice).div(1e18);
        console.log("amountToPay", amountToPay);
        return amountToPay;
    }

    // Sell tokens back to the marketplace
    function sellGLDToken(uint256 _amountOfToken) public {
        require(
            gldToken.balanceOf(msg.sender) >= _amountOfToken,
            "invalid amount of token"
        );
        sellerCount = sellerCount + 1;
        uint256 priceToPayToUser = calculateTokenPrice(_amountOfToken);
        gldToken.safeTransferFrom(msg.sender, address(this), _amountOfToken);
        (bool success, ) = payable(msg.sender).call{value: priceToPayToUser}(
            ""
        );
        require(success, "Transaction Failed");
        emit TokenSold(msg.sender, _amountOfToken, priceToPayToUser);
    }

    // Owner can withdraw excess tokens from the contract
    function withdrawTokens(uint256 _amount) public onlyOwner {
        require(gldToken.balanceOf(address(this)) >= _amount, "Out of balance");
        gldToken.safeTransfer(msg.sender, _amount);
        emit TokensWithdrawn(msg.sender, _amount);
    }

    // Owner can withdraw accumulated Ether from the contract
    function withdrawEther(uint256 _amount) public onlyOwner {
        require(address(this).balance >= _amount, "Invalid Ether amount");
        (bool success, ) = payable(msg.sender).call{value: _amount}("");
        require(success, "Transaction Failed");
    }
```

```
}

//buyerCount = 5
//sellerCount = 1
//markedDemand ratio =
//buyerCount.mul(1e18).div(sellerCount) = 510^18 /1 = 510^18
//adjustedRatio = (510^18 + 110^18)/2 = (6 * 10^18)/2 = 3 * 10^18
// newTokenPrice = //(2 * 10^16 * 3 * 10^18) / 10^18 = (6 * 10^34)/10^18 = 6*10^16
wei = 0.06 ether
```

VOTING INTEGRATED WITH ERC20

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.26;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";

contract Vote {
    //first entity
    struct Voter {
        string name;
        uint256 age;
        uint256 voterId;
        Gender gender;
        uint256 voteCandidateId; //candidate id to whom the voter has voted
        address voterAddress; //EOA of the voter
    }

    //second entity
    struct Candidate {
        string name;
        string party;
```

```
        uint256 age;
        Gender gender;
        uint256 candidateId;
        address candidateAddress; //candidate EOA
        uint256 votes; //number of votes
    }

    //third entity
    address public electionCommission;
    address public winner;
    uint256 nextVoterId = 1;
    uint256 nextCandidateId = 1;

    //voting period
    uint256 startTime;
    uint256 endTime;
    bool stopVoting;
    mapping(uint256 => Voter) voterDetails;
    mapping(uint256 => Candidate) candidateDetails;

    IERC20 public gldToken;

    enum VotingStatus {
        NotStarted,
        InProgress,
        Ended
    }
    enum Gender {
        NotSpecified,
        Male,
        Female,
        Other
    }

    constructor(address _gldToken) {
        gldToken = IERC20(_gldToken);
        electionCommission = msg.sender; //msg.sender is a global variable
    }

    modifier isVotingOver() {
```

```
        require(
            block.timestamp <= endTime && stopVoting == false,
            "Voting time is over"
        );
        _;
    }

    // end time - 1720540700 - Tuesday, 9 July 2024 21:28:20
    // current time - 1720540861 - Tuesday, 9 July 2024 21:31:01

    modifier onlyCommissioner() {
        require(msg.sender == electionCommission, "Not authuorized");
        _;
    }

    modifier isValidAge(uint256 _age) {
        require(_age >= 18, "not eligible for voting");
        _;
    }

    function registerCandidate(
        string calldata _name,
        string calldata _party,
        uint256 _age,
        Gender _gender
    ) external isValidAge(_age) {
        require(
            isCandidateNotRegistered(msg.sender),
            "You are already registered"
        );
        require(nextCandidateId < 3, "Candidate Registration Full");
        require(
            msg.sender != electionCommission,
            "Election Commision not allowed to register"
        );

        candidateDetails[nextCandidateId] = Candidate({
            name: _name,
            party: _party,
            gender: _gender,
```

```
            age: _age,
            candidateId: nextCandidateId,
            candidateAddress: msg.sender,
            votes: 0
        });
        nextCandidateId++;
    }

    function isCandidateNotRegistered(address _person)
        private
        view
        returns (bool)
    {
        for (uint256 i = 1; i < nextCandidateId; i++) {
            if (candidateDetails[i].candidateAddress == _person) {
                return false;
            }
        }
        return true;
    }

    function getCandidateList() public view returns (Candidate[] memory) {
        Candidate[] memory candidateList = new Candidate[](nextCandidateId - 1);
//initialize an empty array of length = `nextCandidateId - 1`
        for (uint256 i = 0; i < candidateList.length; i++) {
            candidateList[i] = candidateDetails[i + 1];
        }
        return candidateList;
    }

    function isVoterNotRegistered(address _person) private view returns (bool) {
        for (uint256 i = 1; i < nextVoterId; i++) {
            if (voterDetails[i].voterAddress == _person) {
                return false;
            }
        }
        return true;
    }

    function registerVoter(
```

```
        string calldata _name,
        uint256 _age,
        Gender _gender
    ) external isValidAge(_age) {
        require(isVoterNotRegistered(msg.sender), "You are already registered");

        voterDetails[nextVoterId] = Voter({
            name: _name,
            age: _age,
            voterId: nextVoterId,
            gender: _gender,
            voteCandidateId: 0,
            voterAddress: msg.sender
        });
        nextVoterId++;
    }

    function getVoterList() public view returns (Voter[] memory) {
        uint256 lengthArr = nextVoterId - 1;
        Voter[] memory voterList = new Voter[](lengthArr);
        for (uint256 i = 0; i < voterList.length; i++) {
            voterList[i] = voterDetails[i + 1];
        }
        return voterList;
    }

    function castVote(uint256 _voterId, uint256 _candidateId)
        external
        isVotingOver
    {
        require(gldToken.balanceOf(msg.sender) > 0, "not allowed");
        require(block.timestamp >= startTime, "Voting has not started yet");
        require(
            voterDetails[_voterId].voteCandidateId == 0,
            "You have already voted"
        );
        require(
            voterDetails[_voterId].voterAddress == msg.sender,
            "You are not authourized"
        );
```

```
        require(
            _candidateId >= 1 && _candidateId < 3,
            "Candidate Id is not correct"
        );
        voterDetails[_voterId].voteCandidateId = _candidateId; //voting to
_candidateId
        candidateDetails[_candidateId].votes++; //increment _candidateId votes
    }

    function setVotingPeriod(
        uint256 _startTimeDuration,
        uint256 _endTimeDuration
    ) external onlyCommissioner {
        require(
            _endTimeDuration > 3600,
            "_endTimeDuration must be greater than 1 hour"
        );
        startTime = 1720799550 + _startTimeDuration; //_startTimeDuration = 3600 ,
_endTimeDuration = 3600
        endTime = startTime + _endTimeDuration;
    }

    function getVotingStatus() public view returns (VotingStatus) {
        if (startTime == 0) {
            return VotingStatus.NotStarted;
        } else if (endTime > block.timestamp && stopVoting == false) {
            return VotingStatus.InProgress;
        } else {
            return VotingStatus.Ended;
        }
    }

    function announceVotingResult() external onlyCommissioner {
        uint256 max = 0;
        for (uint256 i = 1; i < nextCandidateId; i++) {
            if (candidateDetails[i].votes > max) {
                max = candidateDetails[i].votes;
                winner = candidateDetails[i].candidateAddress;
            }
        }
```

```
    }

    function emergencyStopVoting() public onlyCommissioner {
        stopVoting = true;
    }
}
```